Product Requirements Document (PRD): VaultAuth SSO

1. Header Information

Field	Value
Project Name	**VaultAuth SSO** (Enterprise Single Sign-On Portal)
Project Track & ID	`PST_PD_154` / Product Development
Batch / Level	2026 Level 4 (Stretch)
Duration	4 Weeks
Team Composition	**2-Student Agile Engineering Team** • **Apoorva Bhardwaj**: Frontend Lead & Security Client Architecture ([vaultauth-frontend](https://github.com/ABhardwaj28/vaultauth-frontend)) • **Shaurya Jain**: Backend Lead & Identity API Architecture ([vaultauth-backend](https://github.com/ProPlays2312/vaultauth-backend))
PRD Version	**v2.0 (Optimized Full-Stack Enterprise Architecture & REST Contract)**
Repository Links	• Frontend: `https://github.com/ABhardwaj28/vaultauth-frontend` • Backend: `https://github.com/ProPlays2312/vaultauth-backend`

2. Executive Overview

VaultAuth SSO is a zero-trust single sign-on (SSO) and identity access management (IAM) platform designed for modern enterprise workforces. It bridges client-side cryptographic hashing with a high-throughput backend authentication service, providing:

1. **Zero-Knowledge Secret Transmission:** Passwords are pre-hashed client-side using standard SHA-256 before network transmission or local persistence.

2. **Multi-Factor Verification (2FA OTP):** Primary password verification triggers an automated 6-digit mock OTP challenge.

3. **Active Session Telemetry & 1-Click Revocation:** Complete visibility over active devices, IP addresses, and session lifecycles, enabling enterprise administrators to revoke compromised sessions globally in under 30 seconds.

4. **Pluggable Dual-Mode Architecture:** The frontend functions seamlessly out-of-the-box in standalone **Mock/Local Mode** (using `localStorage` and `sample-data.json`) and instantly

transitions to **Live Backend Mode** connecting to `vaultauth-backend` via a clean RESTful service adapter.

- **Industry:** Cybersecurity / Cloud Identity
- **Theme:** Zero-Trust Identity & Access Management (IAM)
- **One-Line Pitch:** *"For enterprise security teams seeking turnkey single sign-on without the latency and bloat of massive legacy identity providers, VaultAuth SSO delivers client-hashed verification, simulated 2FA OTP gating, and active session revocation in under 30 seconds."*

---

3. Problem Statement & Root Cause Analysis

3.1 The Vulnerability Landscape

- **Cleartext & Reversible Credential Leaks:** Entry-level web portals frequently store credentials in cleartext or use weak client hashing algorithms. In the event of client-side inspection or data layer compromise, enterprise passwords are exposed directly.
- **Absence of Secondary Authentication Gates:** Single-factor authentication allows leaked passwords to directly result in complete account compromise and lateral network movement.
- **Zombie & Hijacked Active Sessions:** When employees switch devices or connect over insecure public Wi-Fi networks, active sessions linger indefinitely. Traditional applications provide no real-time auditability or remote kill-switch for administrators to sever compromised active sessions.

3.2 Target Remediation (Solved When)

1. **Client-Side Pre-Hashing:** Passwords are transformed into 64-character SHA-256 hexadecimal digests using `crypto.subtle` prior to local caching or HTTP transmission.

2. **Two-Factor Authentication (2FA) Gate:** Entering valid primary credentials transitions the state into `OTP_REQUIRED`; tokens are only issued upon 6-digit OTP verification.

3. **Session Revocation Management:** Real-time session monitoring with device fingerprinting and instant 1-click token invalidation.

4. **Brute-Force Lockout Defense:** Consecutive failed password attempts automatically trigger a mandatory 5-minute timed account lockout.

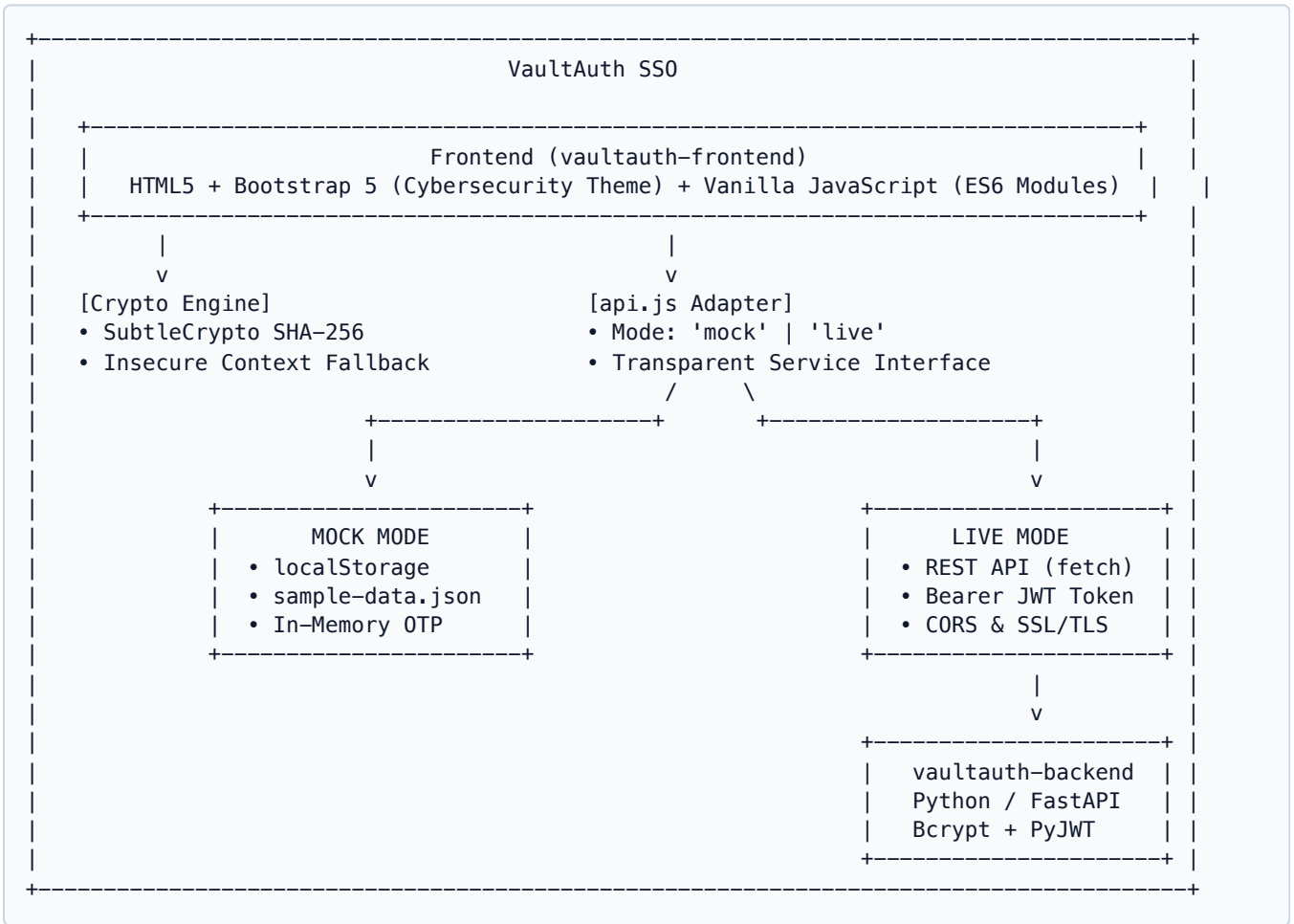
---

4. Users, Roles & RBAC Matrix

The system enforces strict Role-Based Access Control (RBAC) across two core personas:

Capability	End User (Employee)	Enterprise Admin
Select Active Role (Demo Quick-Sign)	Yes	Yes
Register New Account (Email + Pre-Hashed Password)	Yes	Yes
Primary Credential Verification (Email + Password)	Yes	Yes
Authenticate with 2FA Mock OTP	Yes	Yes
View Own Active Sessions & Digital ID Card	Yes	Yes
Revoke Own Active Session (Sign Out Device)	Yes	Yes
Revoke Any Third-Party User Session	**No (403 Forbidden)**	**Yes**
Inspect Global Security Audit Logs	**No (Hidden View)**	**Yes**
Reset SSO Portal State (Demo Directory Seeding)	Yes (Demo)	Yes (Demo)

5. System Architecture & Dual-Mode Adapter



5.1 Pluggable Adapter Pattern

To avoid tightly coupling the frontend to `localStorage`, the frontend architecture routes all requests through `api.js`:

- **Offline / Mock Mode:** Seeds initial state from `sample-data.json`, persists mutations to `localStorage`, and generates in-memory OTP codes. Ideal for standalone grading, offline demos, and client UI testing.
- **Live Backend Mode:** Simply changing `API_CONFIG.mode = 'live'` routes all calls to `http://localhost:8000/api/v1` (or production host), sending standard JSON payloads and processing backend JWT session tokens.

---

6. Scope & Feature Boundaries

6.1 Must Have (MVP — Graded)

- **User Registration:** Form captures email and password, enforces password complexity, client-hashes password with SHA-256, and creates account record.
- **Primary Authentication:** Compares email and computed SHA-256 digest against credentials.
- **Mock 2FA OTP Flow:** Generates 6-digit OTP passcode with 3-minute expiration; validates OTP input before granting session.
- **Session Lifecycle:** Generates session tokens upon successful 2FA; persists active session state.
- **Logout & Self-Revocation:** Clears current active session token and returns to login gate.
- **Responsive Bootstrap 5 Layout:** Tailored cybersecurity UI responsive across mobile (375px) to desktop (1440px).

6.2 Good to Have (Distinction Level)

- **Account Lockout Guard:** 3 consecutive failed password attempts trigger a 5-minute timed lock on the target account (`BR-03`).
- **Active Session Manager:** Lists active login sessions with device details, IP address, and 1-click revocation buttons (`FR-05`, `FR-06`).
- **Security Audit Log:** Immutable event log tracking `LOGIN_SUCCESS_2FA`, `FAILED_LOGIN_ATTEMPT`, `SESSION_REVOKED`, and `ACCOUNT_LOCKED` (`FR-07`).
- **Demo Evaluation Toast:** Floating non-intrusive badge displaying current OTP for frictionless grading (`Risk Mitigation`).
- **Reset Demo Directory:** One-click reset button restoring initial admin/user data from `sample-data.json`.

6.3 Stretch (Self-Study & Advanced)

- **Corporate Digital ID Card:** Visual employee credential card displaying identity photo/avatar, roles, department, and active session status.
- **WebAuthn / FIDO2 Concept Card:** Reading-only interactive card explaining public-key cryptography, biometric authenticators (TouchID/FaceID), and hardware security keys.
- **React Native (Expo Go) Companion App:** Optional 2-screen mobile companion app consuming the same backend API.

6.4 Out of Scope

- Real production SMS/carrier gateways (e.g., Twilio) or live email SMTP servers.
- Live Okta/Auth0 OAuth2 identity brokering.
- Physical YubiKey USB hardware parsing.

---

7. Functional Requirements

ID	Requirement Name	Priority	Specification & Acceptance Criteria
FR-01	**Client-Side SHA-256 Hashing**	Must	Registration and login forms must compute a 64-character SHA-256 hexadecimal hash using `crypto.subtle.digest()` prior to transmission or persistence. Passwords must never exist in plaintext in storage or console logs.
FR-02	**Primary Authentication Flow**	Must	Verifies email and computed password hash. If correct, transitions UI to `OTP_REQUIRED` state without issuing session token. If incorrect, increments failed attempts counter.
FR-03	**Mock 2FA OTP Generation & Validation**	Must	Generates a 6-digit numeric passcode with 3-minute TTL. Inputting matching 6 digits validates challenge, creates an active session token, and navigates to role dashboard.
FR-04	**3-Strike Account Lockout**	Good	3 consecutive failed password attempts set user account status to `LOCKED` for 5 minutes (300 seconds). Prevents authentication attempts until lockout expiration.
FR-05	**Active Session Telemetry**	Good	Detects and records client device (`navigator.userAgent`), simulated/real IP, timestamp, and active status. Displays list of active user sessions.
FR-06	**Granular Session Revocation**	Good	End Users can revoke their own sessions; Enterprise Admins can revoke any user session. Terminating a session immediately invalidates the session token.
FR-07	**Security Audit Logger**	Good	Logs timestamps, email, event type, and IP for security actions. Enterprise Admins can view audit logs. All values are XSS-sanitized before DOM insertion.
FR-08	**Dual-Mode Data Persistence**	Must	Seamlessly switches between `localStorage` (seeded by `sample-data.json`) and live REST API calls to `vaultauth-backend` . Preserves state across browser reloads.

8. Technology Stack

8.1 Frontend Client (`vaultauth-frontend`)

- **Markup & Structure:** HTML5 (Semantic, Accessible, WAI-ARIA compliant).
- **Styling & Design System:** Bootstrap 5.3 + Custom Cybersecurity Theme (Dark/Light mode, high-contrast badges, glowing security status pills).
- **Application Logic:** Vanilla JavaScript (ES6+ Modules, zero heavy framework dependencies).

- **Cryptography:** Web Cryptography API (`window.crypto.subtle`) with pure JS SHA-256 fallback for non-secure testing contexts.
- **Hosting / Deployment:** GitHub Pages / Vercel.

8.2 Backend Service (`vaultauth-backend`)

- **Language & Runtime:** Python 3.10+
- **Web Framework:** FastAPI (or Flask with Flask-RESTful)
- **Hashing & Security:** Passlib / Bcrypt for backend salting of client hashes; PyJWT for signed session tokens.
- **Data Storage:** SQLite (Development) / PostgreSQL (Production) / In-Memory Seed fallback.
- **Hosting / Deployment:** Render / Railway / Localhost.

---

9. Data Model & Entity Schemas


```
{
  "schemaVersion": 2,
  "users": [
    {
      "id": "usr_101",
      "email": "employee@vaultauth.io",
      "passwordHash": "8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918",
      "fullName": "Jane Doe",
      "department": "Engineering",
      "role": "user",
      "status": "ACTIVE",
      "failedAttempts": 0,
      "lockoutUntil": null
    },
    {
      "id": "usr_999",
      "email": "admin@vaultauth.io",
      "passwordHash": "8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918",
      "fullName": "Alex Rivera",
      "department": "SecOps",
      "role": "admin",
      "status": "ACTIVE",
      "failedAttempts": 0,
      "lockoutUntil": null
    }
  ],
  "sessions": [
    {
      "sessionId": "sess_8820",
      "userId": "usr_101",
      "device": "Chrome Browser (macOS)",
      "ipAddress": "192.168.1.45",
      "loginTime": "2026-10-30T10:00:00Z",
      "lastActive": "2026-10-30T10:25:00Z",
      "status": "ACTIVE"
    }
  ],
  "otpChallenges": [
    {
      "userId": "usr_101",
      "otpCode": "849201",
      "expiresAt": "2026-10-30T10:03:00Z",
      "attemptsRemaining": 3
    }
  ],
  "auditLogs": [
    {
      "id": "log_001",
      "timestamp": "2026-10-30T10:00:00Z",
      "email": "employee@vaultauth.io",
      "event": "LOGIN_SUCCESS_2FA",
      "ip": "192.168.1.45"
    }
  ]
}
```

10. RESTful API & Service Contract

All endpoints exchange JSON payloads. In live mode, authenticated routes expect an `Authorization: Bearer` header.

10.1 Endpoints Specification

1. Register User

- **Endpoint:** `POST /api/v1/auth/register`
- **Access:** Public
- **Request Body:**

`json

{

"email": "newuser@vaultauth.io",

"passwordHash": "64_char_sha256_hex_digest",

"fullName": "John Smith",

"role": "user"

}

`

- **Success Response (201 Created):**

`json

{

"success": true,

"user": { "id": "usr_102", "email": "newuser@vaultauth.io", "role": "user" }

}

`

- **Error Codes:** `400 Bad Request` (`ERR_DUPLICATE_EMAIL`, `ERR_WEAK_PASSWORD`, `ERR_INVALID_HASH`).

2. Authenticate User (Primary Factor)

- **Endpoint:** `POST /api/v1/auth/login`
- **Access:** Public
- **Request Body:**

`json

{

"email": "employee@vaultauth.io",

"passwordHash": "64_char_sha256_hex_digest"

}

`

- **Success Response (200 OK):**

`json

{

"status": "OTP_REQUIRED",

"userId": "usr_101",

"tempToken": "tmp_challenge_token",

"expiresInSeconds": 180

}

`

- **Error Codes:** `401 Unauthorized` (`ERR_INVALID_CREDENTIALS`), `423 Locked` (`ERR_ACCOUNT_LOCKED`, returns remaining lockout seconds).

3. Verify 2FA Mock OTP (Secondary Factor)

- **Endpoint:** `POST /api/v1/auth/verify-otp`
- **Access:** Public (Temporary Challenge Token)
- **Request Body:**

`json

{

"userId": "usr_101",

"otpCode": "849201",

"tempToken": "tmp_challenge_token",

"deviceInfo": "Chrome Browser (macOS)"

}

`

- **Success Response (200 OK):**

`json

{

"success": true,

"sessionToken": "jwt_session_token_xyz",

"user": { "id": "usr_101", "email": "employee@vaultauth.io", "role": "user", "fullName": "Jane Doe" },

"sessionId": "sess_8820"

}

`

- **Error Codes:** `401 Unauthorized` (`ERR_INVALID_OTP` , returns attempts remaining), `410 Gone` (`ERR_OTP_EXPIRED`).

4. Get Active Sessions

- **Endpoint:** `GET /api/v1/sessions`
- **Access:** Authenticated (Bearer Token)
- **Query Params (Admin only):** `?userId=usr_101` (optional filter)
- **Success Response (200 OK):**

`json

[

{

"sessionId": "sess_8820",

"userId": "usr_101",

"userEmail": "employee@vaultauth.io",

"device": "Chrome Browser (macOS)",

"ipAddress": "192.168.1.45",

"loginTime": "2026-10-30T10:00:00Z",

"isCurrentSession": true

}

]

`

5. Revoke Session

- **Endpoint:** `POST /api/v1/sessions/{sessionId}/revoke`
- **Access:** Authenticated (Owner of session OR Admin)
- **Success Response (200 OK):**

`json

{

"success": true,

"revokedSessionId": "sess_8820",

"message": "Session successfully terminated"

}

`

- **Error Codes:** `403 Forbidden` (`ERR_FORBIDDEN` if non-admin attempts to revoke another user's session), `404 Not Found` (`ERR_SESSION_NOT_FOUND`).

6. Retrieve Audit Logs

- **Endpoint:** `GET /api/v1/audit/logs`
- **Access:** Admin Only (Bearer Token)
- **Success Response (200 OK):**

`json

[

{

"id": "log_001",

"timestamp": "2026-10-30T10:00:00Z",

"email": "employee@vaultauth.io",

"event": "LOGIN_SUCCESS_2FA",

"ip": "192.168.1.45"

}

]

`

- **Error Codes:** `403 Forbidden` (`ERR_FORBIDDEN`).

---

11. Business Rules & Operational Policies

- **BR-01 (Zero Cleartext Storage):** Passwords must never be stored in plaintext in memory, storage, or logs. Secret values must be computed as a SHA-256 hexadecimal string client-side before any storage or transit.
- **BR-02 (Mandatory 2FA Gate):** Valid primary credentials must never directly yield an active session token. Successful password validation must navigate to a 2FA OTP prompt with an active 3-minute timer.
- **BR-03 (3-Strike Account Lockout):** If a user enters 3 consecutive invalid passwords, the account status transitions to `LOCKED` for exactly 300 seconds (5 minutes). Lockout timestamps must persist to prevent browser refresh evasion.
- **BR-04 (Session Revocation Boundary):** Enterprise Admins possess authority to terminate any active session across all accounts; End Users are strictly bounded to terminating their own active sessions.

- **BR-05 (OTP Lifespan & Max Retries):** An OTP challenge is valid for 180 seconds. Three invalid OTP entries invalidate the challenge, terminating the temporary authentication session and forcing the user back to the primary login screen.
- **BR-06 (SubtleCrypto Degradation Fallback):** When running in unsecure HTTP contexts (e.g. `file://` or non-localhost IPs) where `window.crypto.subtle` is undefined, the application must seamlessly execute a pure JS SHA-256 fallback algorithm without failing or halting.

---

12. Security Engineering & Data Safety

- **XSS Defense:** All values rendered into DOM components (emails, device strings, IP addresses, audit event descriptions) must be strictly escaped using HTML character entity encoding.
- **Token Storage Discipline:** Session tokens are stored in `sessionStorage` or in-memory variables to minimize persistent disk exposure, with standard `Authorization: Bearer` request decoration.
- **CORS Security:** Backend API must configure strict CORS headers allowing only designated frontend origins during development and production.
- **Role Tampering Prevention:** The frontend UI conditionally hides admin panels, and the backend re-validates cryptographic token claims on every privileged operation (`GET /audit/logs` , `POST /sessions/*/revoke`).

---

13. Error Handling & Edge Cases

Scenario	System & UI Behavior	HTTP Code (Live)
Incorrect Password	Increments `failedAttempts` . Shows inline alert: *"Invalid credentials. Attempt N of 3."*	`401 Unauthorized`
Account Locked (3 Fails)	Blocks input. Displays countdown timer: *"Account locked due to consecutive failures. Try again in MM:SS."*	`423 Locked`
Incorrect OTP Code	Alerts *"Invalid 2FA code. N attempts remaining."* Clears OTP input and focuses on the first digit.	`401 Unauthorized`
Expired OTP Code	Alerts *"2FA passcode expired. Please re-authenticate."* Returns user to login screen.	`410 Gone`
Unauthorized Session Kill	Shows alert *"Permission Denied: Only Enterprise Admins can revoke external user sessions."*	`403 Forbidden`
Storage Full / Quota Exceeded	Captures quota error, saves state to memory, and renders warning banner.	N/A (Client Fallback)

---

14. 4-Week Milestone Roadmap

Phase	Frontend Deliverables (Apoorva Bhardwaj)	Backend Deliverables (Shaurya Jain)
Week 1: Setup & Contracts	• GitHub repo initialization (`vaultauth-frontend`). • Responsive Bootstrap 5 layout & cyber theme. • `sample-data.json` & identity schema definition. • Pluggable `api.js` adapter setup.	• GitHub repo initialization (`vaultauth-backend`). • Python virtualenv & dependency configuration. • Pydantic identity schemas & models. • SQLite / in-memory database setup.
Week 2: Core Auth & 2FA	• SubtleCrypto SHA-256 pre-hashing + JS fallback. • Split 6-digit OTP input component. • User registration & login views. • Grading evaluation toast helper.	• `POST /auth/register` with backend hashing. • `POST /auth/login` primary credential check. • In-memory 2FA challenge & OTP generation. • `POST /auth/verify-otp` JWT token issuance.
Week 3: Sessions & Revocation	• Active session dashboard with device telemetry. • 1-click session revocation UI. • Persistent 5-minute account lockout counter. • Security audit log table (Admin only).	• `GET /sessions` with Bearer auth validation. • `POST /sessions/{id}/revoke` endpoint. • Account lockout state persistence. • `GET /audit/logs` with RBAC verification.
Week 4: Integration & Launch	• Connect `api.js` live adapter to backend API. • Cross-browser testing (Chrome, Safari, Firefox). • Vercel / GitHub Pages deployment. • Comprehensive documentation & demo video.	• CORS & production deployment (Render/Railway). • Integration verification test scripts. • Backend README & API docs (Swagger/OpenAPI). • Final evaluation demo walkthrough.

15. Grading Rubric & Evaluation Criteria

- **Functional Authenticity (30%):** Complete user registration, client-side SHA-256 hashing, 2FA OTP verification, active session tracking, and 1-click revocation.
- **Security & Zero-Trust Posture (25%):** Absolute zero plaintext passwords stored/logged, account lockout enforcement after 3 failed attempts, role-based boundary enforcement, XSS-safe rendering.

- **Code Architecture & Decoupling (20%):** Clean modular separation in `vaultauth-frontend` (`app.js`, `crypto.js`, `api.js`, `ui.js`), adapter pattern enabling dual-mode operation (Mock vs. Live).
- **Enterprise UI/UX (15%):** Modern cybersecurity design, split 6-digit OTP box UX, mobile and desktop responsive views, evaluation helper toast.
- **Repository & Documentation (10%):** Clean commit history, comprehensive READMEs across both repositories, detailed PRD alignment.

---

16. Risks, Mitigations & Evaluator Tools

Risk Identified	Likelihood	Impact	Built-in Mitigation
SubtleCrypto Blocked in Insecure Contexts	Medium	High	Dual-engine cryptographic wrapper: automatically detects missing `window.crypto.subtle` and falls back to standard pure JS SHA-256 algorithm.
Grading Friction Inspecting Mock OTPs	Medium	Low	**Grading Helper Toast:** An opt-in evaluation toast displays the active 6-digit passcode directly on screen for instantaneous verification by mentors.
Corrupted / Wiped LocalStorage State	Low	Medium	**Reset Demo Directory Button:** Injects clean default admin (`admin@vaultauth.io`) and user (`employee@vaultauth.io`) state from `sample-data.json`.
CORS Failure during Integration	Medium	High	Backend includes permissive CORS middleware for development (`*` or `localhost:3000`), and frontend includes proxy fallback.

---

17. Project Deliverables

1. **Frontend GitHub Repository:** `https://github.com/ABhardwaj28/vaultauth-frontend` (Clean commits, modular code, comprehensive README).

2. **Backend GitHub Repository:** `https://github.com/ProPlays2312/vaultauth-backend` (API endpoints, Pydantic models, auth logic).

3. **Live Deployed Web Portal:** Hosted on GitHub Pages or Vercel.

4. **Project Requirements Document (PRD):** This document (`PST_PD_154_VaultAuth_SSO.md` and compiled PDF).

5. **Demonstration Walkthrough:** End-to-end video demonstrating registration, client hashing, 2FA OTP flow, 3-attempt account lock, and admin session revocation.

---

18. References & Industry Standards

- **NIST SP 800-63B:** Digital Identity Guidelines — Authentication and Lifecycle Management.
- **W3C Web Cryptography API:** `https://www.w3.org/TR/WebCryptoAPI/`
- **FIDO2 / WebAuthn Standards:** World Wide Web Consortium (W3C) Passwordless Recommendations.
- **OWASP Top 10 Application Security Risks:** Identification and Authentication Failures (A07:2021).
- **Okta & Duo Mobile Zero-Trust Architecture:** Modern Enterprise Session Revocation Models.

---

19. Team Sign-off & Approval

Role	Name	Date	Status
Frontend Lead	Apoorva Bhardwaj	Week 1	**[Approved]**
Backend Lead	Shaurya Jain	Week 1	**[Approved]**
Course Mentor	Assigned Mentor	Week 1	**[Pending Review]**